HIGH PERFORMANCE PYTHON

Поза межами Python

Шлях до швидкодії на C

Векторизація, JIT-компіляція, Cython та Zero-copy

Сумець Світлана

2026

Коли Python не вистачає

> Зручність Python має свою ціну: інтерпретатор не завжди встигає за вимогами сучасних навантажень.

Вузькі місця

Повільні цикли, зайві копії даних та фрагментація пам'яті гальмують обробку великих масивів.

Наша мета

Використати підходи C (швидкість, контроль пам'яті) без втрати зручності та читабельності Python.

Інструментарій

Векторизація, Dask, JIT (Numba), Cython та Zero-copy.

PERFORMANCE GAP

Ми не відмовляємось від Python, ми лише змінюємо підхід до критичних обчислень.

Мислення масивами

> Векторизація - це зміна парадигми: замість того, щоб керувати кожним елементом окремо, ми віддаємо наказ підмножині даних одночасно.

Масова обробка

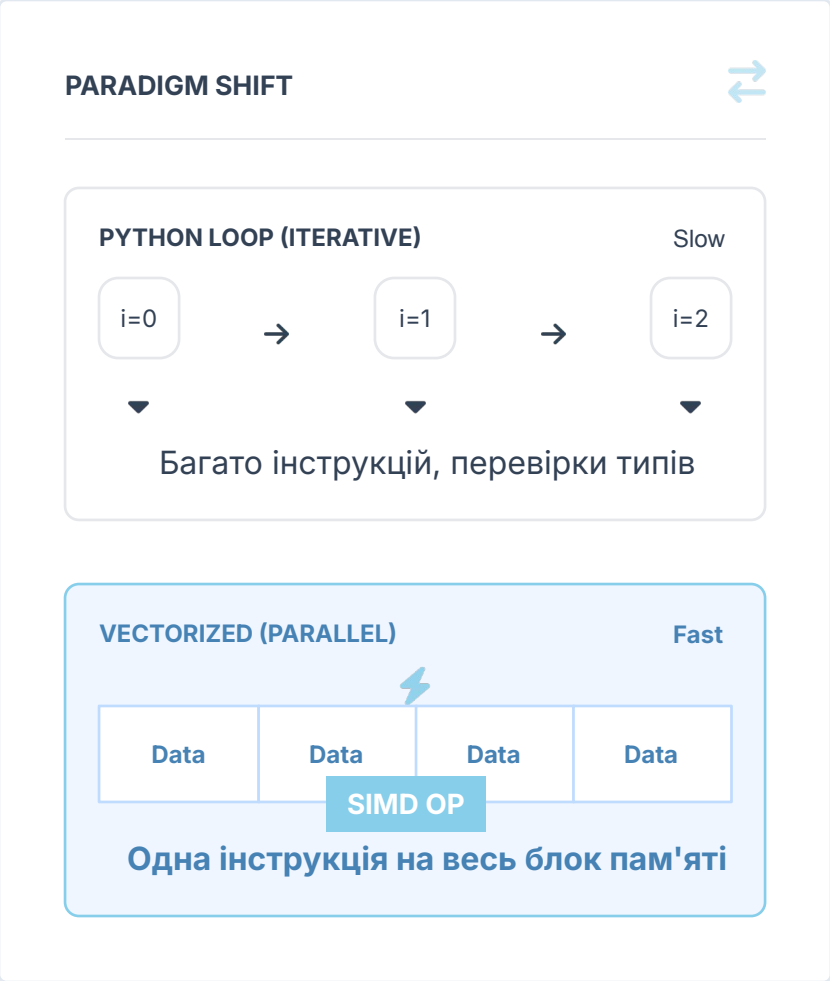
Одна SIMD-інструкція процесора виконується над цілим блоком даних, уникаючи накладних витрат на інтерпретацію циклу.

Кеш-локальність

Дані лежать у пам'яті щільним блоком (contiguous memory), що дозволяє процесору ефективно передбачати та завантажувати їх.

Інструментарій

Основні бібліотеки для векторних обчислень: NumPy, Pandas, JAX, Polars.

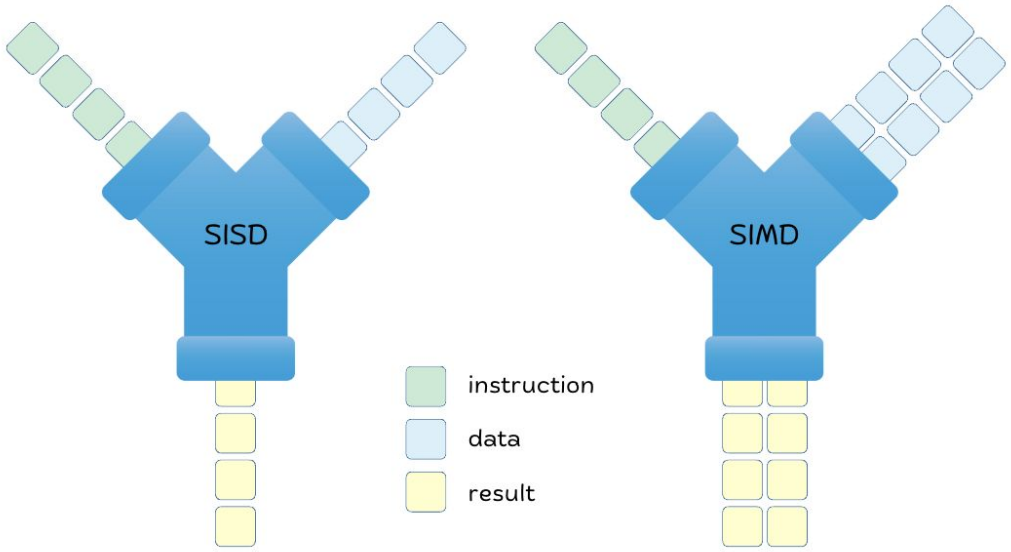


SIMD на рівні заліза

Паралелізм даних

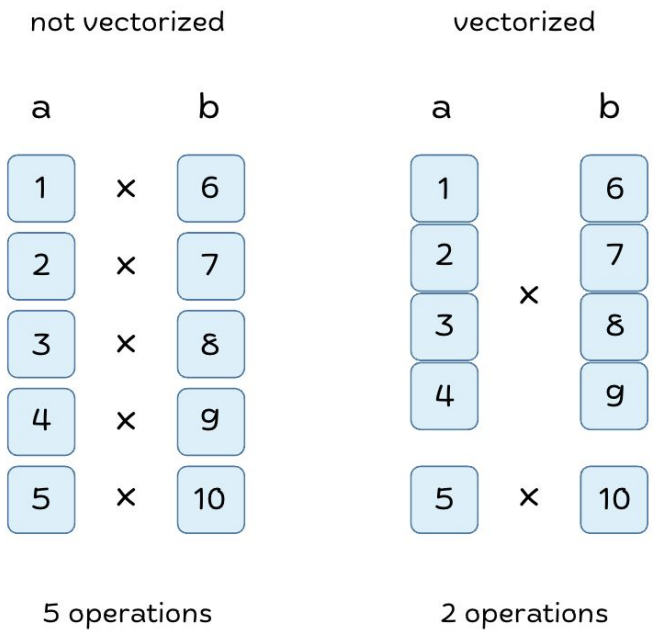
Процесор виконує одну інструкцію (наприклад, додавання) одразу над чотирма 64-бітними числами (float).

SISD vs SIMD



Що більше даних, то менший відсоток "хвостів" (скалярної обробки) і вищий загальний приріст швидкості.

ВЕКТОРНИЙ ПІДХІД



5 елементів — 2 такти

4 items | 1

4 items (SIMD) + 1 item (Scalar)

12 елементів — 3 такти

4 | 4 | 4

3 x SIMD operations

Цикл vs Векторизація

Класичний Цикл

Pure Python

ЧАС ВИКОНАННЯ

0.6136 с

ПАМ'ЯТЬ

315.23 MB

Overhead на створення кожного int/float об'єкта та динамічну типізацію.

vs

NumPy Векторизація

C-Level Optimization

ЧАС ВИКОНАННЯ

0.0146 с

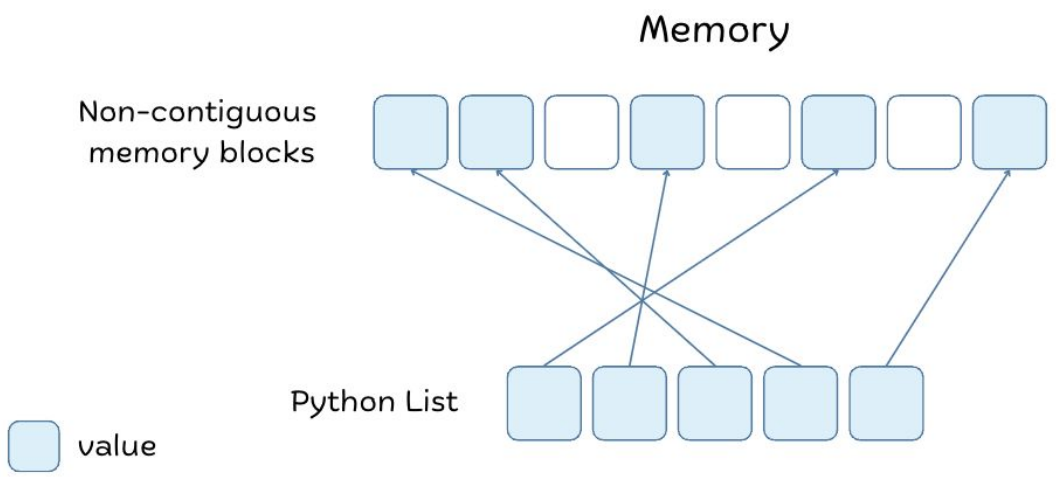
~40x Швидше

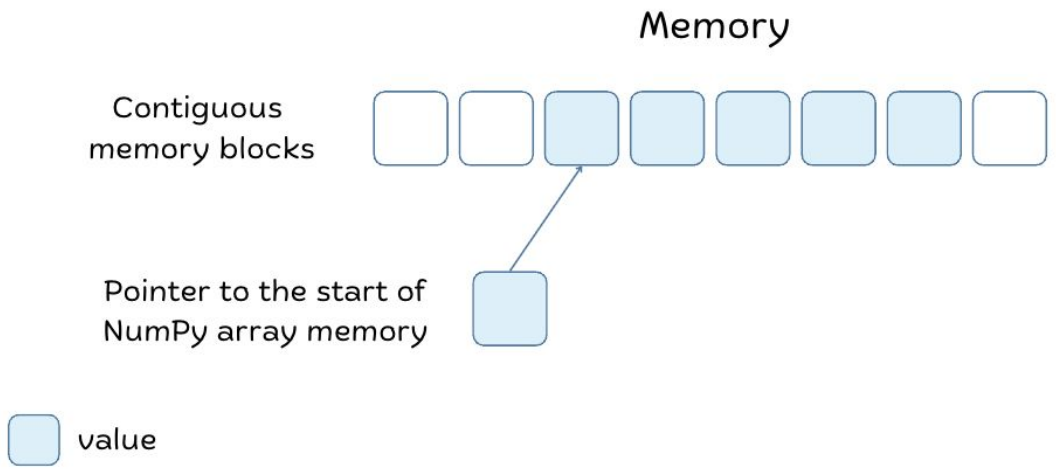
ПАМ'ЯТЬ

77.28 MB

~4x Менше пам'яті

SIMD-інструкції обробляють дані пачками, без зайвих обгорток Python.



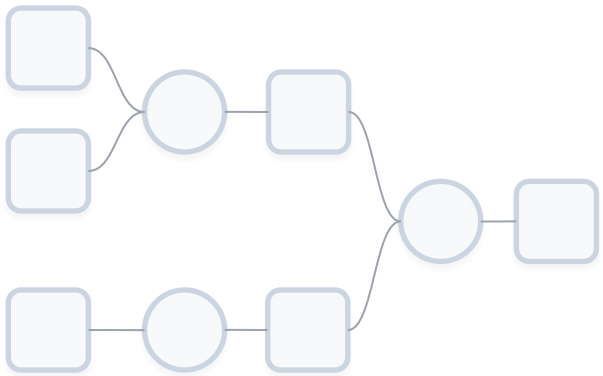


Dask для великих даних

1. РОЗБИТТЯ НА ЧАНКИ

- Data Array
- Dask DataFrame
- Dask Bag
- Dask Delayed
- Futures

2. ПОБУДОВА ГРАФУ (DAG)



3. ПАРАЛЕЛЬНЕ ВИКОНАННЯ

Single-machine (threads, processes, synchronous)

Distributed

Lazy Evaluation

Dask не виконує обчислення одразу. Він будує план (граф задач) і запускає його тільки при виклику .compute().

ОСНОВНІ ПЕРЕВАГИ

- Не вантажить все в RAM
- Масштабується (Cluster)
- API як у Pandas/NumPy
- Усі ядра CPU задіяні

Pandas vs Dask: Практична різниця

Pandas DataFrame

In-Memory

Монолітне завантаження (Single Block)

FULL DATASET

ЧАС ВИКОНАННЯ

17.335 с

ПІК RAM

7.2 GB

Завантажує весь файл у RAM. Ризик Out Of Memory при великих обсягах.

VS

Dask DataFrame

Lazy & Parallel

Обробка частинами (Partitions)

ЧАС ВИКОНАННЯ

9.560 с

~2x Швидше

ПІК RAM

4.7 GB

~35% Економія пам'яті

Використовує ядра CPU паралельно. Працює з даними, більшими за RAM.

Важливо: Для малих даних Pandas швидший через відсутність overhead на планування задач (DAG).

JIT-компіляція: прискорюємо те, що вже є

> Just-In-Time (JIT) компіляція перетворює частини Python-коду на машинний код прямо під час виконання, оминаючи повільну інтерпретацію.

Без інтерпретатора

Найбільше гальмо Python - цикл інтерпретації (Eval Loop). JIT генерує нативний бінарний код, який CPU виконує напряму.

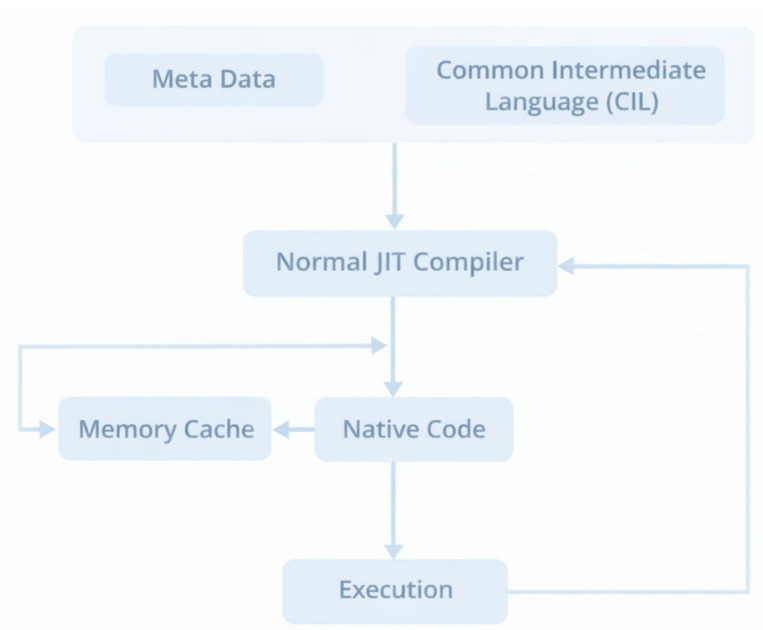
Спеціалізація (Warm-up)

Перший запуск повільний (аналіз типів + компіляція). Усі наступні - миттєві ("швидкий шлях") для цих типів даних.

Інструменти

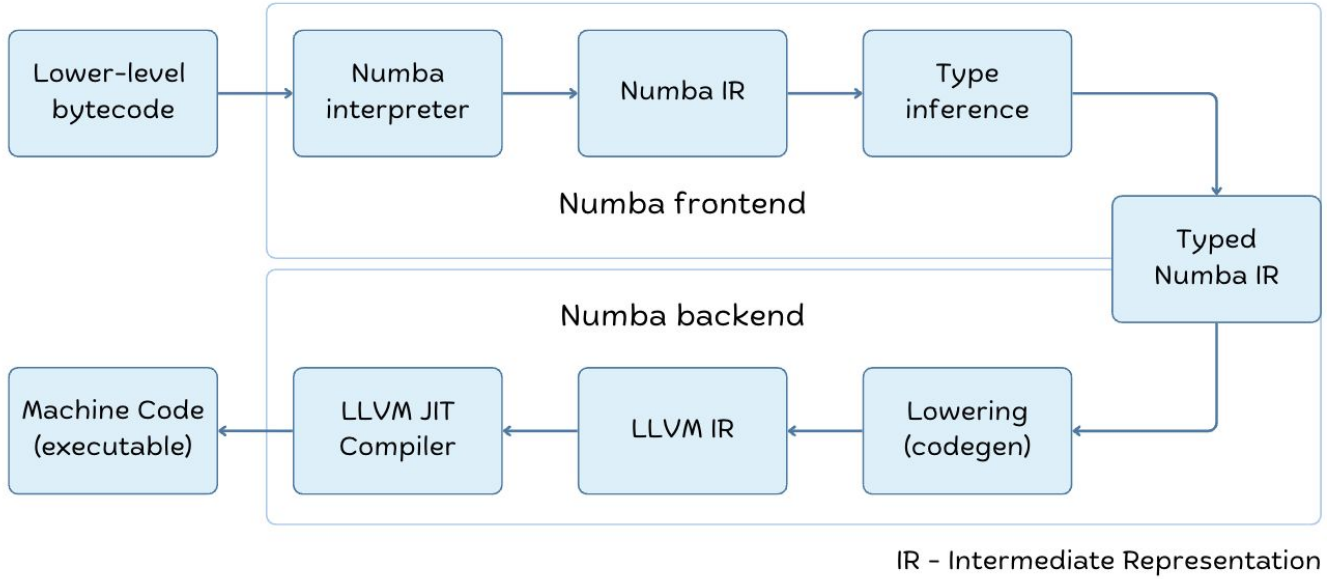
Numba (@njit) для числових циклів та NumPy. JAX (XLA) для тензорних обчислень та автоматичної векторизації.

МЕХАНІЗМ WARM-UP



✓ Ідеально для важких циклів for, які неможливо векторизувати через складну логіку.

Numba: процес компіляції



Функція аналізується, компілюється у високопродуктивний машинний код за допомогою LLVM, а під час наступних викликів виконується вже без інтерпретації Python, напряму на рівні CPU. Завдяки цьому функції з інтенсивними обчисленнями - зокрема ті, що працюють з циклами, умовами та масивами - отримують значне пришвидшення без зміни початкового Python-коду.

Вартість першого виклику

Перший запуск функції завжди повільніший, оскільки Numba повинна пройти всі етапи компіляції: аналіз байт-коду, виведення типів та генерацію машинного коду через LLVM.

Спеціалізація (Specialization)

Numba створює різні версії однієї функції для різних типів вхідних даних.

func(int)
Версія A

func(float)
Версія B

Кешування

Наступні виклики оминають етапи аналізу та використовують готовий машинний код із кешу, досягаючи швидкості C/C++.

Python vs Numba: Матриця відстаней

Базовий Python

NumPy Implementation

Подвійний вкладений цикл $O(n^2)$.
Інтерпретатор обробляє кожну операцію.
Арифметика над Python-об'єктами.
Неможливо векторизувати без зміни підходу.

ЧАС ВИКОНАННЯ

2.86 с

ПАМ'ЯТЬ

~30 MB

Навіть з NumPy операції над окремими елементами в циклі залишаються повільними через overhead інтерпретатора.

VS

Numba Optimized

@njit Decorator

1-й запуск (Компіляція)
Аналіз типів + LLVM компіляція — **0.37 с**

Наступні запуски (Кеш)
Виконання машинного коду — **0.0075 с**

ПРИСКОРЕННЯ

~380x

Радикальний приріст продуктивності

ПАМ'ЯТЬ

~30 MB

Без додаткових витрат пам'яті

Після "розігріву" (warm-up) код виконується зі швидкістю оптимізованого C/C++, використовуючи ту ж пам'ять.

JAX: векторизація + XLA

XLA COMPILATION PIPELINE

INPUT

Python Func
NumPy API

JAX Expression
Intermediate Graph

XLA Compiler
Optimization

FAST

Execution
Machine Code

Tracing
Відстеження операцій та побудова графу залежностей незалежно від Python.

Fusion
Об'єднання багатьох дрібних операцій в одне оптимізоване ядро (Kernel).

Vectorization
Автоматичне розпаралелювання (SIMD) через vmap.

Ефективність JIT

1-й запуск (Компіляція) 0.16s

overhead

Наступні запуски (Run) 0.002s

🚀 ~80x Faster

LOW MEMORY OVERHEAD

КЛЮЧОВІ МОЖЛИВОСТІ

Автоматична векторизація Функція vmap автоматично перетворює код для роботи з батчами даних.

Backend Agnostic
Той самий код працює на CPU, GPU та TPU без змін.

Cython: зручність Python, швидкість C

BUILD PIPELINE

Source Code .pyx
Python синтаксис + C типи

`cythonize (Transpile)`

C Source .c
Оптимізований C-код з викликами CPython API

`GCC / MSVC (Compile)`

Extension Module .so / .pyd
Нативний машинний код, готовий до імпорту

cython_example.pyx

```
def fib_cython(int n):
    cdef int a = 0
    cdef int b = 1
    cdef int i
    for i in range(n):
        a, b = b, a + b
    return a
```

Static Typing

AOT-компіляція (Ahead-Of-Time)

На відміну від JIT, код компілюється один раз перед запуском. Це дає максимальну стабільність і відсутність "розігріву" (warm-up).

Контроль пам'яті та C-типи

Використання cdef дозволяє працювати з "чистими" C-змінними, обходячи overhead об'єктів PyObject. Ідеально для циклів (tight loops).

Числова ефективність

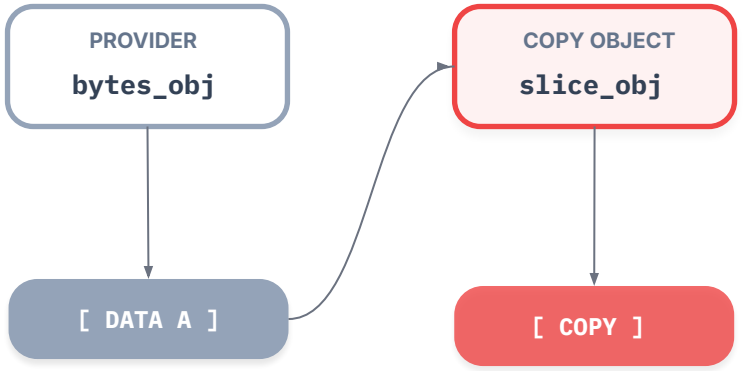
Максимальний виграш у математичних розрахунках, де Python витрачає час на розпакування (unboxing) чисел.

Zero-copy: buffer protocol та memoryview

Стандартне копіювання

bytes / str slicing

slice = obj[0:5]



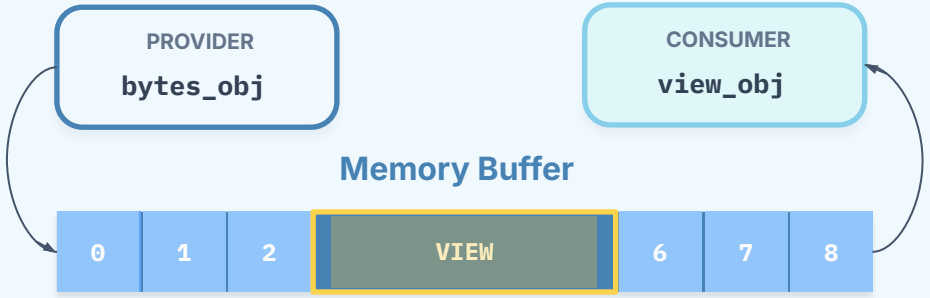
New Allocation
Пам'ять дублюється

RAM Spike

Zero-copy підхід

buffer protocol / memoryview

Shared Memory Buffer



memoryview "дивиться" в той самий буфер

Zero Allocations
Використання існуючих даних

0 MB Overhead

Обробка великих файлів | I/O Пайплайни (Socket/File) | Обробка зображень | NumPy / Pandas Interop

mmap: Файли як Пам'ять

Фізичний Диск (SSD/HDD)
Файл: large_dataset.bin (100 GB)

BLK1 | BLK2 | BLK3 | ... | BLKn

Page Cache (RAM)
Кешування лише активних сторінок

Lazy Loading

Page1 | Page2 | Empty | ... | Empty

Virtual Memory Map
Прямий доступ без syscall read()

mmap_obj

VAddr1 | VAddr2 | Unmap | ... | Unmap

memoryview | NumPy | Pandas

МИТТЄВИЙ ДОСТУП

Не потрібно чекати завантаження всього файлу. ОС підвантажує сторінки (4KB) лише коли програма звертається до них.

ЕКОНОМІЯ RAM

Файл 100 GB може бути відкритий на машині з 4 GB RAM. Пам'ять використовується лише для "активного вікна".

ZERO-COPY ІНТЕГРАЦІЯ

NumPy та Pandas можуть створювати масиви напряму поверх mmap-буфера без жодного копіювання байтів.

ПРИКЛАД ВИКОРИСТАННЯ

```
with open("big.bin", "r+b") as f:
  mm = mmap.mmap(f.fileno(), 0)
  view = memoryview(mm)
```

Порівняння структур даних (1M записів)

СТАНДАРТНИЙ ПІДХІД

List[dict]

267.45 MB

↑ Base Line

HASH MAP STRUCTURE

"id"	→	1001
"name"	→	"test"
"age"	→	25

Дублювання ключів

ОПТИМІЗАЦІЯ №1

Pandas DataFrame

71.42 MB

↓ -73% Пам'яті

COLUMNAR (NUMPY BACKED)

col:int	[1001, 1002...]
col:obj	["a", "b", "c"]
col:int	[25, 26, 27...]

Block Manager Overhead

МАКСИМАЛЬНА ЕФЕКТИВНІСТЬ

PyArrow Table

28.5 MB

↓ -89% Пам'яті

CONTIGUOUS + ZERO-COPY

Int64 Buffer

1	2	3	...

String Buffer

A	B	C	...

Zero-Copy | No Overhead

Тест на 1 мільйоні записів. Arrow використовує компактне бінарне представлення, ідентичне в пам'яті та на диску.

Джерело: Internal Benchmarks

Висновки: інженерія без жертв

01

Векторизація та пам'ять

Уникайте циклів Python. Використовуйте суцільні блоки пам'яті (NumPy, Arrow) для максимальної кеш-локальності та використання SIMD.

02

Zero-copy підхід

Найшвидша операція — та, якої не було. Використовуйте memoryview та mmap для роботи з великими даними без зайвого копіювання.

03

JIT для "гарячих" циклів

Там, де потрібна складна логіка, яку важко векторизувати, використовуйте компіляцію на льоту (Numba, JAX) для досягнення швидкості C.

04

Cython для hot-spots

Статична типізація та C-розширення для найбільш критичних вузлів, де потрібен повний контроль над пам'яттю.

05

Спочатку вимірюйте

Інтуїція може підвести. Завжди профілюйте код перед оптимізацією, щоб знайти реальні вузькі місця.

Філософія Python

"Ми не відмовляємося від зручності Python. Ми використовуємо його як клей для високопродуктивних компонентів."

Що почитати?

- The limits of Python vectorization
- Extending and Embedding the Python Interpreter
- Buffer Protocol
- Memoryview
- Arrow Columnar Format
- What is NumPy?
- Pandas. Enhancing performance
- Numba. Performance Tips
- Python C extensions